



2025summer 实验报告

蔡中仪 202200460081

2025 年 8 月 13 日

目录

1	project1	2
1.1	题目	2
1.2	SM4 原理	2
1.2.1	密钥扩展 (Key Schedule)	3
1.3	优化	3
1.3.1	指令优化	3
1.3.2	T-Table 优化	4
2	project2	4
2.1	题目	4
2.2	代码原理	4
2.3	基础模型	5
2.4	优化	7
3	project3	9
3.1	题目	9
3.2	环境配置	9
3.3	代码原理	12
3.4	基础模型	13
4	project4	13
4.1	题目	13
4.2	SM3 原理	13
4.2.1	基本流程	13
4.2.2	消息扩展	14
4.3	基础模型	15
4.4	优化	15
5	project5	16
5.1	题目	16
5.2	SM2 原理	17
5.3	基础模型	18
5.4	优化	18
5.4.1	第一次优化	18
5.4.2	第二次优化	19
6	project6	19
6.1	题目	19
6.2	代码原理	19
6.3	基础模型	21
6.4	优化原理	21
6.5	优化效果	22

1 project1

1.1 题目

做 SM4 的软件实现和优化

1.2 SM4 原理

SM4 是一个分组对称加密算法，用于中国的无线局域网标准。与 AES 类似，SM4 是一种对称加密算法，用这个主要因为它安全性高、计算效率高、适合硬件实现等特点。

基本信息

属性	描述
算法类型	分组加密，对称加密
分组长度	128 位 (16 字节)
密钥长度	128 位 (16 字节)
加密轮数	32 轮
加密结构	类似 Feistel 结构 (非标准)

表 1: SM4 基本参数

具体流程

SM4 使用了类 Feistel 的结构 (但不是标准 Feistel 结构)，加密过程如下：

- 明文输入会被划分为 4 个 32 位字： X_0, X_1, X_2, X_3
- 加密执行 32 轮迭代，每轮更新：

$$X_{i+4} = X_i \oplus F(X_{i+1}, X_{i+2}, X_{i+3}, rk_i)$$

- 其中 rk_i 为第 i 轮的轮密钥， F 为轮函数

轮函数 F 的定义

在 SM4 里，轮函数 F 定义为：

$$F(B, rk) = T(B \oplus rk)$$

其中变换 T 包含以下两步：

- 非线性变换 (使用 S 盒替代)：

$$\tau(A) = \text{SBox}(A)$$

每个字节通过一个固定的 8×8 的 S-盒查表替代。

- 线性变换 L ：

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

最终：

$$T(A) = L(\tau(A))$$

其中的 $\lll$ 表示的是循环左移操。

1.2.1 密钥扩展 (Key Schedule)

初始密钥为 128 位 (4 个 32 位字): MK_0, MK_1, MK_2, MK_3 , 结合固定常量 FK_i 和轮常量 CK_i 生成 32 轮密钥 $rk_0 \sim rk_{31}$ 。

密钥扩展公式如下:

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

其中:

$$T'(A) = L'(\tau(A)) \quad , \quad \text{其中 } L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

加密与解密

- 加密过程使用轮密钥 rk_0 到 rk_{31} 。
- 解密过程使用轮密钥 rk_{31} 到 rk_0 (逆序)。
- 轮函数 F 相同, 因此解密无需额外结构, 便于硬件实现。

最终密文输出为:

$$Y = (X_{35}, X_{34}, X_{33}, X_{32})$$

总结起来

明文分组为 X_0, X_1, X_2, X_3 , 每轮变换如下:

$$\text{for } i = 0 \text{ to } 31 : X_{i+4} = X_i \oplus F(X_{i+1}, X_{i+2}, X_{i+3}, rk_i)$$

最终输出为: $X_{35}, X_{34}, X_{33}, X_{32}$ (逆序)

1.3 优化

1.3.1 指令优化

下面这几个优化都是非常常见的思路, 这里就不再一个个赘述, 所以总在一起写了, 针对原始 SM4 实现, 我为了提升运行效率与降低内存占用, 主要进行了以下优化:

1. **减少函数嵌套**: 将频繁调用的小函数直接内联到调用点, 减少调用栈深度和参数传递开销, 同时利用局部变量缓存中间值, 降低从外部作用域取值的成本, 从而减少函数调用次数并提升执行速度。
2. **减少内存使用**: 复用已分配的缓冲区, 避免在加解密过程中频繁创建新的列表或字节串对象, 从而降低垃圾回收压力和内存峰值占用。
3. **函数手动展开, 提高速度**: 利用 SM4 32 轮迭代固定的特性, 将部分关键函数 (如 T 变换、S 盒替换) 直接写成连续运算代码, 并对循环进行部分或完全展开, 减少循环与函数调用的开销。

4. **使用局部变量代替列表，减少内存复制**：将频繁访问的列表元素拆分为独立的局部变量（如 x_0, x_1, x_2, x_3 ），利用 Python 的 `LOAD_FAST` 指令提升访问速度，避免索引运算和不必要的内存复制。
5. **避免数组**：减少临时数组（列表、元组）的生成，改用独立变量存储中间结果，必要时通过结构化数据或字节序列聚合，减少对象创建与管理的开销。
6. **使用 bytearray 避免字符串拼接开销**：原实现若使用不可变的 `bytes` 进行数据拼接，会导致每次操作都重新分配内存并复制数据。改为使用可变字节序列 `bytearray`，通过索引或 `extend()` 方法直接写入，最后一次性转换为 `bytes`，显著降低内存分配与数据复制成本。

优化后可以正确运行：

```
>>> = RESTART: G:\Desktop\2025summer\project1\sm4.py
原文: b'Hello SM4 2025summer'
密文: cfcf2df140cba57199aa6d4494003841cfa4a88f27b001356af6b90418a28bb5
解密: b'Hello SM4 2025summer'
>>>
```

1.3.2 T-Table 优化

T-Table 优化原理本质上是“把原来需要多步执行的 S 盒替换 + 线性变换组合运算，提前计算好结果放到查找表里”，这样在加密过程中只需一次查表即可得到最终结果，从而减少指令数、提升速度，可以看到成功加速了

```
===== RESTART: G:\Desktop\2025summer\project1\sm4.py =====
数据大小: 0.02 KB
原始实现耗时: 0.000055 s
T-Table耗时: 0.000029 s
加速比: 1.88x
加密结果一致: True
```

2 project2

2.1 题目

编程实现图片水印嵌入和提取（可依托开源项目二次开发），并进行鲁棒性测试，包括但不限于翻转、平移、截取、调对比度等

2.2 代码原理

我实现了一个基于离散余弦变换（DCT）的数字图像水印嵌入与提取系统，并对水印的鲁棒性进行了测试。其核心思想及步骤如下：

核心思想

- 将宿主图像转换到 YUV 色彩空间，分别对三个通道（Y、U、V）进行处理。
- 将水印图像二值化，要求水印尺寸不超过宿主图像分块后的尺寸。
- 宿主图像按 8×8 块划分，对每个块进行二维 DCT 变换。
- 利用两个随机密钥向每个块的 DCT 系数中特定位置（代码中为最后一列）添加扰动，实现水印嵌入。
- 提取时通过计算与两个密钥的相关性，判断水印位是 0 还是 1。
- 对嵌入后的图像施加多种攻击（噪声、旋转、裁剪、对比度变化、JPEG 压缩、模糊），测试水印的鲁棒性。

代码思路

1. 读取宿主图和水印图，将水印图二值化。
2. 将宿主图转换为 YUV 空间。
3. 分别对 Y、U、V 通道：
 - 初始化 WatermarkProcessor;
 - 对宿主通道做分块 DCT;
 - 嵌入水印，进行逆 DCT 恢复空域图像;
 - 提取水印并保存恢复结果。
4. 将三个通道的水印嵌入图合并，显示原图、水印、嵌入图及提取水印。
5. 对嵌入图施加多种攻击，测试水印提取效果，评估鲁棒性。

通过对图像块的 DCT 频域系数微调实现水印的不可见嵌入。利用两个随机密钥分别对应水印的 0、1 位，增强了安全性和鲁棒性。通过计算相关性提取水印，提高了对常见图像处理攻击的抵抗能力。基于 YUV 三通道分别嵌入的设计，也提升了信息容量和抗干扰性能。

2.3 基础模型

采用基于二维离散余弦变换（2D-DCT）的数字图像水印技术。其基本流程包括：

1. 将宿主图像 I 按照块大小 $N \times N$ 进行划分，通常取 $N = 8$ ，得到若干图像块 $B_{i,j}$ ，其中 i, j 分别表示块在图像中的行列索引。
2. 对每个图像块 $B_{i,j}$ 施加二维 DCT 变换，得到频域系数矩阵：

$$D_{i,j} = \text{DCT}(B_{i,j})$$

这里 $D_{i,j}$ 是一个 $N \times N$ 的实数矩阵，包含该块的频率分量。

3. 设计二值水印矩阵 W ，其尺寸满足 $W \leq \frac{H}{N} \times \frac{W}{N}$ ，与图像块数量对应。

4. 选择两个长度为 N 的随机密钥向量 $\mathbf{k}_0, \mathbf{k}_1 \in R^N$ ，分别代表水印的 0 和 1。
5. 根据水印二值 $w_{i,j} \in \{0, 1\}$ ，对 $D_{i,j}$ 的特定频率分量（如最后一列）进行调整：

$$D'_{i,j}[:, N] = D_{i,j}[:, N] + \alpha \cdot \mathbf{k}_{w_{i,j}}$$

其中， α 是嵌入强度系数，控制水印强弱，保证水印不可见但可检测。

6. 对修改后的频域块 $D'_{i,j}$ 进行逆 DCT，恢复空域水印块 $B'_{i,j}$ ，最终拼接得到带水印的图像 I' ：

$$B'_{i,j} = \text{IDCT}(D'_{i,j}), \quad I' = \bigcup_{i,j} B'_{i,j}$$

水印提取过程为：

1. 对带水印图像 I' 同样进行 $N \times N$ 分块及 DCT，得到 $D''_{i,j}$ 。
2. 提取 $D''_{i,j}$ 的对应频率分量 $\mathbf{v}_{i,j}$ （如最后一列）。
3. 计算其与两个密钥的归一化相关系数：

$$\rho_0 = \frac{(\mathbf{v}_{i,j} - \bar{\mathbf{v}})^T (\mathbf{k}_0 - \bar{\mathbf{k}}_0)}{\|\mathbf{v}_{i,j} - \bar{\mathbf{v}}\| \cdot \|\mathbf{k}_0 - \bar{\mathbf{k}}_0\|}, \quad \rho_1 = \frac{(\mathbf{v}_{i,j} - \bar{\mathbf{v}})^T (\mathbf{k}_1 - \bar{\mathbf{k}}_1)}{\|\mathbf{v}_{i,j} - \bar{\mathbf{v}}\| \cdot \|\mathbf{k}_1 - \bar{\mathbf{k}}_1\|}$$

4. 依据相关系数大小判定水印位：

$$\hat{w}_{i,j} = \begin{cases} 1, & \rho_1 > \rho_0 \\ 0, & \text{otherwise} \end{cases}$$

该基础模型利用 DCT 频域的局部块特性，有效嵌入信息且保证水印的隐蔽性和一定的抗攻击能力。

一开始加水印结果如下：



2.4 优化

为增强水印系统的性能和鲁棒性，我进行了以下优化设计：

多通道独立嵌入 将宿主图像由 RGB 转换为 YUV 色彩空间，分别对 Y（亮度）、U、V（色度）三个通道进行水印嵌入。由于人眼对亮度更敏感，而色度通道抗干扰能力更强，三通道综合利用可提高水印的容量和鲁棒性。每个通道的水印嵌入过程独立，提取时分别处理再合成水印图像。

随机密钥设计与相关判别 采用两个随机向量作为水印 0 和 1 的代表密钥，密钥生成如下：

$$\mathbf{k}_0, \mathbf{k}_1 \sim \mathcal{N}(0, 1)$$

密钥的随机性增加了系统安全性，难以通过盲目修改频域系数伪造水印。相关判别法可有效抑制噪声干扰，提升提取准确率。

嵌入强度调节 嵌入强度 α 作为关键参数，直接影响水印的隐蔽性与鲁棒性。通过实验调节 α ，实现视觉质量与水印稳定性的权衡。代码中支持灵活调整，满足不同应用需求。

多种攻击模拟测试 设计了多种常见攻击模型：

- **高斯噪声**：模拟传输噪声。
- **旋转变换**：模拟几何变形。
- **裁剪与缩放**：模拟图像部分丢失。
- **对比度调整**：模拟亮度 and 对比度变化。
- **JPEG 压缩**：模拟有损压缩。
- **模糊处理**：模拟图像模糊。

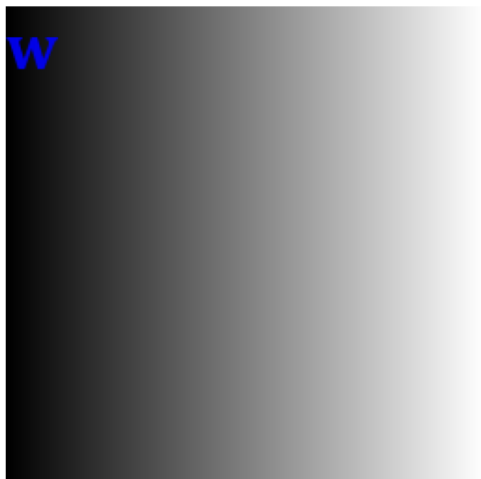
对这些攻击进行系统测试，保证水印能在恶劣条件下依然被正确提取。

块处理与计算效率 采用基于块的 DCT 分解策略，便于并行处理，提高计算效率。算法在图像块上局部操作，减少计算复杂度，适合实时和大规模图像水印处理。

综合上述优化，系统有效提升了水印的隐蔽性、安全性及抗攻击能力，适用于数字版权保护和内容认证等实际场景。

优化后结果如图：

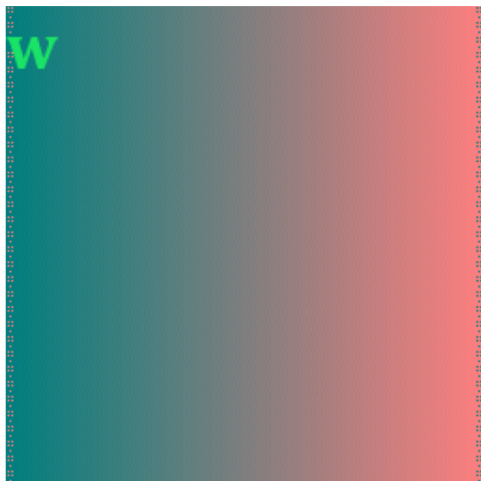
原图



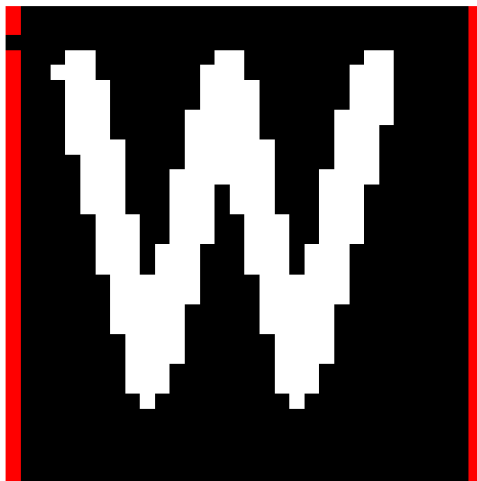
水印



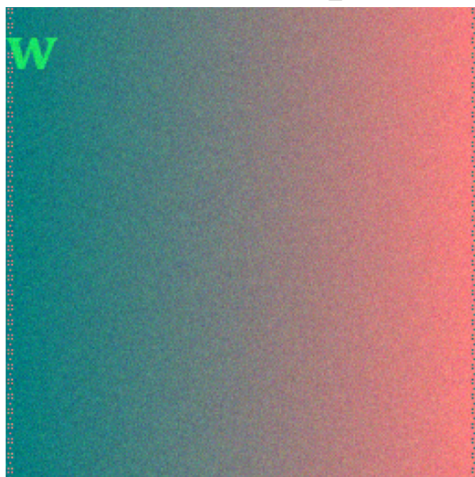
嵌入图



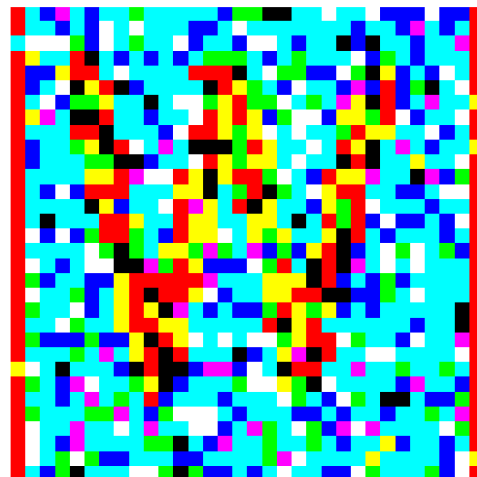
提取水印



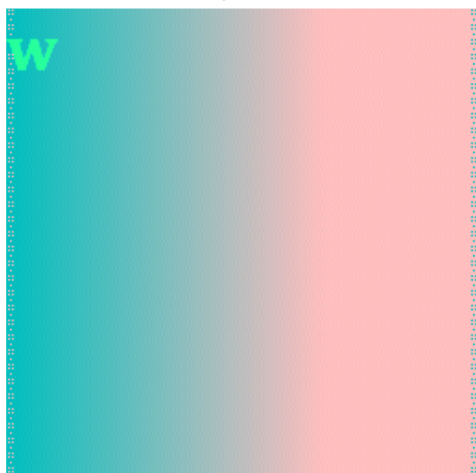
攻击后图像 (gaussian_noise)



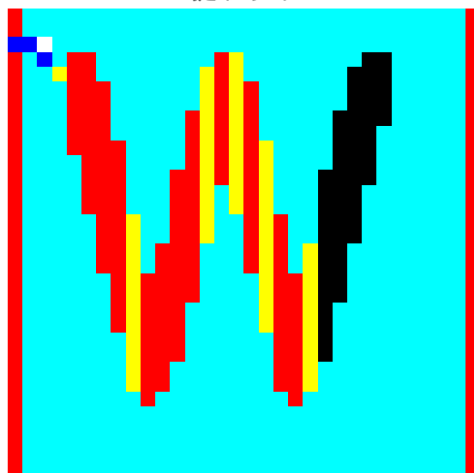
提取水印



攻击后图像 (contrast)



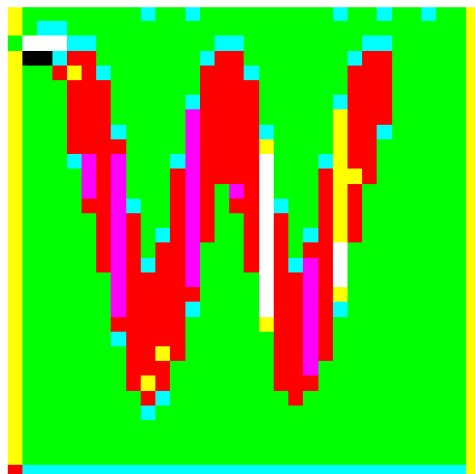
提取水印



攻击后图像 (blur)



提取水印



3 project3

3.1 题目

用 circom 实现 poseidon2 哈希算法的电路

要求:

- (1) poseidon2 哈希算法参数参考参考文档 1 的 Table1, 用 $(n,t,d)=(256,3,5)$ 或 $(256,2,5)$
- (2) 电路的公开输入用 poseidon2 哈希值, 隐私输入为哈希原象, 哈希算法的输入只考虑一个 block 即可。
- (3) 用 Groth16 算法生成证明

3.2 环境配置

直接下载就行:

[下载](#)

下载

[circom Linux 二进制文件](#)[circom macOS 二进制文件](#)[circom Windows 二进制文件](#)

先运行指令：

```
1 curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh
```

这个看样子是安装了一些依赖项
可以正确下载：

Rust is installed now. Great!

To get started you may need to restart your current shell.

This would reload your **PATH** environment variable to include

Cargo's bin directory (`$HOME/.cargo/bin`).

To configure your current shell, you need to source the corresponding **env** file under `$HOME/.cargo`.

This is usually done by running one of the following (note the leading DOT):

```
. "$HOME/.cargo/env"          # For sh/bash/zsh/ash/dash/pdksh
```

```
source "$HOME/.cargo/env.fish" # For fish
```

```
source "$nu.home-path/.cargo/env.nu" # For nushell
```

```
[08/10/25]seed@VM:~/Desktop$
```

参考网址为: <https://docs.circom.io/getting-started/installation/important-deprecation-note>
安装好了 (和其他项目不是一个环境, 这是在 ubuntu 上跑的):

```
[08/10/25]seed@VM:~/.../circom$ circom --help
circom compiler 2.2.2
IDEN3
Compiler for the circom programming language

USAGE:
  circom [FLAGS] [OPTIONS] [--] [input]

FLAGS:
  --rlcs                Outputs
                        the constraints in rlcs format
  --sym                 Outputs
                        witness in sym format
  --wasm                Compile
                        s the circuit to wasm
  --json                Outputs
                        the constraints in json format
  --wat                 Compile
                        s the circuit to wat
```

安装 snarkjs:

```
1 npm install -g snarkjs
```

就可以了

3.3 代码原理

Circom 是一个专门用来编写和生成零知识证明电路 (zk-SNARK circuits) 的高级语言和工具链。

生成 witness:

```
1 node poseidon_hash2_js/generate_witness.js poseidon_hash2_js/poseidon_hash2.wasm in
```

3.4 基础模型

```
[08/13/25]seed@VM:~/.../circomlib-master$ circom poseidon_hash2.circom --r1cs --wasm --sym
template instances: 72
non-linear constraints: 243
linear constraints: 274
public inputs: 0
private inputs: 2
public outputs: 1
wires: 520
labels: 771
Written successfully: ./poseidon_hash2.r1cs
Written successfully: ./poseidon_hash2.sym
Written successfully: ./poseidon_hash2_js/poseidon_hash2.wasm
Everything went okay
[08/13/25]seed@VM:~/.../circomlib-master$
```

4 project4

4.1 题目

sm3 的软件实现与优化跟 SM4 一样用 C 语言来做然后不断改进效率

4.2 SM3 原理

SM3 设计思想类似于 SHA-256，用于生成固定长度（256 位）的消息摘要。具备抗碰撞、抗原像和抗第二原像攻击能力。

4.2.1 基本流程

SM3 输入任意长度的消息，输出固定长度的 256 位摘要。主要流程如下：

- 消息填充 (Padding)
- 消息分组 (每组 512 位)
- 消息扩展 (生成 $W_0 \sim W_{67}$ 及 $W'_0 \sim W'_{63}$)
- 迭代压缩函数 (64 轮)
- 最终摘要输出

其整体结构与 SHA-256 类似，但在消息扩展与压缩函数方面有所不同，从而增强了安全性。

4.2.2 消息扩展

将每 512 位分组划分为 16 个初始 32 位字：

$$W_0, W_1, \dots, W_{15}$$

然后扩展为 68 个字：

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \ll 15)) \oplus (W_{j-13} \ll 7) \oplus W_{j-6}, \quad 16 \leq j \leq 67$$

其中 P_1 为非线性置换函数：

$$P_1(X) = X \oplus (X \ll 15) \oplus (X \ll 23)$$

并定义：

$$W'_j = W_j \oplus W_{j+4}, \quad 0 \leq j \leq 63$$

压缩函数 CF

设初始向量为 $V = (A, B, C, D, E, F, G, H)$ ，每一轮压缩如下：

$$SS1 = ((A \ll 12) + E + (T_j \ll j)) \ll 7$$

$$SS2 = SS1 \oplus (A \ll 12)$$

$$TT1 = FF_j(A, B, C) + D + SS2 + W'_j$$

$$TT2 = GG_j(E, F, G) + H + SS1 + W_j$$

其中布尔函数 FF_j 和 GG_j 定义如下：

$$FF_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z) & 16 \leq j \leq 63 \end{cases}$$

$$GG_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (\neg X \wedge Z) & 16 \leq j \leq 63 \end{cases}$$

压缩函数执行 64 轮迭代，每轮使用一个 T_j 常量，不同轮次取值不同。

代码注意的点

- 使用 32 位无符号整数操作（如 `uint32_t`）处理所有数据。
- 所有变换操作依赖位运算（移位、异或、与、或、取反等）。
- 消息扩展阶段预先计算 W_j 与 W'_j ，避免重复计算。
- 压缩函数应使用局部变量表示状态值，减少内存访问。

优化策略

优化类型	优化策略
循环展开	展开 64 轮压缩循环，提升指令级并行性
表达式缓存	将 $A \lll 12$ 等常用表达式缓存到临时变量
消息扩展缓存	提前构造 W_j 和 W'_j 数组，避免重复位移计算
常量预处理	T_j 常量提前定义为静态表，加快访问
向量化优化	使用 SIMD 指令（如 AVX、NEON）并行处理多组消息
内存对齐	确保 32 位字对齐，提升内存访问效率

表 2: SM3 软件实现优化策略

与 SHA-256 的比较

特性	SM3	SHA-256
输出长度	256 位	256 位
消息扩展	68 个字 + 异或扩展 W'	64 个字
布尔函数	两段式复杂布尔函数	简单布尔函数 (Ch, Maj)
安全设计	更复杂的扩展 + 非线性变换	经典结构
应用场景	国家密码应用、签名、区块链等	国际通用，区块链广泛使用

表 3: SM3 与 SHA-256 的对比

SM3 在软件实现中应注意：

- 避免不必要的中间变量重复计算；
- 将压缩函数展开优化，提高流水线效率；
- 对于多消息处理，采用 SIMD 向量化处理；
- 尽量减少内存访问次数，增强缓存友好性。

4.3 基础模型

初始版本可以看到能够正确运行：

4.4 优化

针对 SM3 算法的基础实现，我进行了如下优化，从而能够提升代码性能和可维护性：

1. 减少重复计算和函数调用开销:

- SM3 常量 T_j 在每轮迭代中需左移计算, 预先计算并缓存所有 T_j 值, 避免循环内重复调用旋转函数。
- 将 P_0 、 P_1 函数改为内联函数或宏, 减少函数调用开销。

2. 优化消息扩展:

- 精简消息扩展数组 W 的计算逻辑, 避免多余内存访问, 提升执行效率。

3. 规范变量命名:

- 明确区分哈希状态变量, 避免变量名如 H 和 H_0 造成混淆, 提升代码可读性。

4. 保证类型安全和移位操作规范:

- 所有移位及位操作均限定在 32 位无符号整型范围内, 避免未定义行为。

5. 预先计算常量 T_j :

- SM3 中 T_j 常量在不同区间值不同, 预计算并存储, 主循环直接访问, 提升性能。

6. 代码结构优化:

- 减少重复代码, 模块化设计, 提升代码维护性和扩展性。

进行优化后的结果为:

```
Microsoft Visual Studio 调试器 × + v
原方案输出: 4575b7f5291d3c1fd0494b983047c801d9b3d7a4d9c9d4dd5a326bdd36865322
原方案耗时(us): 45.7
优化方案输出: 4575b7f5291d3c1fd0494b983047c801d9b3d7a4d9c9d4dd5a326bdd36865322
优化方案耗时(us): 7.8
两个方案输出一致 (正确性通过)
标准向量: 66c7f0f462eedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0

G:\Desktop\2025summer\project4\SM3\Debug\SM3.exe (进程 64032)已退出, 代码为 0。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...
```

5 project5

5.1 题目

sm2 的软件实现优化

a). 考虑到 SM2 用 C 语言来做比较复杂, 大家看可以考虑用 python 来做 sm2 的基础实现以及各种算法的改进尝试

b). 20250713-wen-sm2-public.pdf 中提到的关于签名算法的误用分别基于做 poc 验证, 给出推导文档以及验证代码

c). 伪造中本聪的数字签名

5.2 SM2 原理

SM2 是我国国家密码管理局制定的椭圆曲线公钥密码算法标准，基于椭圆曲线离散对数难题 (ECDLP)，实现了密钥交换、数字签名和加密功能。相比 RSA 等传统公钥算法，SM2 在国密系统中具有较高效率和安全性。

a) 软件实现与优化 (Python 为主)

SM2 的核心基于椭圆曲线密码学 (ECC)，使用 C 语言实现复杂且难以调试。考虑到教学、算法验证与安全研究的需求，推荐使用 Python 进行基础版本实现和后续改进尝试。其优势如下：

- **高可读性**：Python 对于大整数支持良好，适合表示 ECC 运算中的多精度运算；
- **模块化实验**：可灵活分离椭圆曲线点运算、签名流程、哈希处理等模块；
- **库支持丰富**：可选用 `ecdsa`、`gmssl`、`tinyec` 等库或自行实现；
- **便于优化**：可以快速尝试不同随机数生成策略、哈希方法或曲线参数；

b) 签名算法误用与漏洞验证 (基于 20250713-wen-sm2-public.pdf)

文献中指出若开发者在使用 SM2 签名时存在如下不当用法，可能导致私钥泄漏或伪造攻击。我们将以 Python 编写 Proof of Concept (PoC) 验证并推导漏洞利用过程。

1) 随机数 k 重用导致私钥泄漏 SM2 签名过程中需生成一次性随机数 k ，若两次使用相同 k ，则攻击者可解出私钥 d ：

$$\begin{aligned}s_1 &= [(1+d)^{-1}(k - r_1d)] \mod n \\ s_2 &= [(1+d)^{-1}(k - r_2d)] \mod n \\ \Rightarrow & \text{攻击者可联立求解出 } d\end{aligned}$$

2) 验签时未正确检测参数边界 若验签方未检查签名参数 r, s 是否落入合法范围内，或未验证公钥是否在曲线上，攻击者可构造非法签名绕过验证。

3) 忽略用户标识 Z 的使用 SM2 签名中，消息哈希为 $e = H(Z||M)$ ，其中 Z 与用户身份、公钥相关。若不使用或错误计算 Z ，则签名安全性大幅下降，可能使签名移植到其他用户下仍有效。

4) 攻击验证方式 我们将编写代码展示上述误用方式，并实现：

- 利用重复 k 值，恢复 SM2 私钥；
- 构造恶意签名，通过不严格验签；
- 比较有无 Z 值参与签名的可验证性；

c) 伪造“中本聪”签名的原理

近年来出现的某些攻击案例展示“伪造中本聪签名”，其原理多基于椭圆曲线签名算法（如 ECDSA）数学结构，尽管实际无法得到私钥，但可构造“通过验签”的伪签名。

基本原理： 以 ECDSA 为例：

- 已知某公钥 $Q = dG$;
- 构造一组 (r, s) 满足：

$$s^{-1}(H(m))G + s^{-1}rQ = R_x$$

- 若该式成立， (r, s) 将通过验签流程；

由于验签算法仅基于数学一致性，而不验证签名过程的“真实性”，攻击者可构造伪签名用于误导舆论或伪造来源。

应用场景：

- 构造“中本聪”对某段声明的签名，诱导其“真伪”判断；
- 针对缺乏时间戳/上下文绑定的签名系统进行社交工程攻击；

安全建议：

- 签名时加入上下文绑定，例如时间戳、来源标识；
- 验签方不仅应校验签名数学结构，还应结合元数据判断上下文；

通过 Python 实现 SM2 可快速验证签名结构、攻击流程与优化策略。结合签名算法误用分析与伪签名构造，可帮助我们更深入理解 SM2 的安全边界与工程实现风险。在实际部署中，应严格按标准实现参数校验与哈希绑定，避免因误用导致私钥泄漏或签名伪造风险。

5.3 基础模型

基础版本可以正确运行：

```
//
= RESTART: G:\Desktop\2025summer\project5\SM2.py
私钥: e030ad79923514adb7da4965355e41987f3cce5b8ae6b50f15c3845ef6b2bd91
公钥: c1f3be08ac3105d2b8236cf2df595e29839646769f69de138f4e76346f7f71298e988cd89e
bcc935f3af3104d1b88020f0e35d518f4db2c1c944eba3d52cc624
签名: 279dd4bf096f4f71ad93d084bcb4cfd50fbc4a9dc97ddcb77b1d1907fbc017837121c2d8eb
7be8630209a61977ffda9fa3d8904c22db74bd01d172a7827f7950
验签结果: True
加密: sXYfZGlz0KiyzY2Z7b0Xfs04gqpDtvGfz0qszvT1TTRm5Esxkf+Fu++jeZDd7mh/DAEab14RgH
3j4goSHmBG/fVv142xiePHBSGqrx+KHEBVU7a+DtcfyJMqvzMMH5gD9ux0yG1Y0XfTI2Wq
解密: b'\xe4\xbd\xa0\xe5\xa5\xbd\xef\xbc\x8cSM2'
>>
```

5.4 优化

5.4.1 第一次优化

第一次优化，我做了：

1. 优化 `generate_keypair` 方法：避免直接调用 `_kg` 等私有方法，改为通过 `CryptSM2` 公共接口的椭圆曲线标量乘法生成公钥，提升可维护性和兼容性。

2. 优化 sign 方法：将随机数 rand_k 的生成方式由 `func.random_hex` 改为 Python 标准库的 `secrets.token_hex`，以增强随机数的安全性，防止弱随机导致的私钥泄露风险。

第一次优化后可以正确输出：

```
私钥: f2776633ee58a66adc4b485ad1eb645adeecdf40e1e23a97c7f81cfa0776c750
公钥: 7875c5d3d0be98b898fbd4db3a1b7e93a2f5ccd7bb3e8defa0a5920a53dfc73abb9b4d350e884fbc065ed889137fa6ab9ceefa3259ace14d8b4892c29ddd5f9
原始消息: 你好, SM2
签名: 2a1b81e9a40aa6360aba155a922ff8f34565d5e55c1c858ff5a60e0bb980e4d5a874c01540406360438fb9a8f24056ba5a7fec53d4fbd996532526215514
验签结果: True
加密结果: LeMh8QUJaz7RnvY7DwAtwdOCZ4tNJvLrpVtye1q0eoZ7AhBtAtZaapn5tcIwA+LVPK55MuMDMKmODT5Ri5FZEKgcAzOZxChC+z8s8kB/pMMH9EAL3B0cntOt/Nqeb6vvyxiANQ8Mo3a/M6A4
解密结果: 你好, SM2
解密成功: True
```

5.4.2 第二次优化

第二次花了一些时间，对多点进行了优化：

- 代码去重与模块化：新增 `sm2_attack_utils.py` 统一实现通用逻辑
- 随机数重用：更新为不依赖 `r` 相等的恢复公式。
- 可预测随机数：采用基于 SM2 签名关系的简化恢复式。

6 project6

6.1 题目

Google Password Checkup 验证

来自刘巍然老师的报告 google password checkup, 参考论文 <https://eprint.iacr.org/2019/723.pdf> 的 section 3.1，也即 Figure 2 中展示的协议，尝试实现该协议，（编程语言不限）。

6.2 代码原理

这个任务实现了论文 Figure 2 所描述的基于 DDH 假设的私有交集求和协议 (Private Intersection-Sum Protocol, Π_{DDH}) 的 Python 演示版本。协议的目标是在两方之间计算交集大小与交集元素值的总和，而不泄露非交集的元素信息。协议包含三轮交互，采用了如下主要密码工具：

- 哈希到群：将标识符映射为群元素，基于安全哈希函数（如 SHA-256）。
- 离散对数困难群：使用模素数乘法群（论文中建议使用椭圆曲线 prime256v1）。
- Paillier 同态加密：支持同态加法，用于安全地累加交集对应的值。

符号说明

- P_1 ：拥有标识符集合 $X = \{v_i\}$ 。
- P_2 ：拥有标识符与整数值对集合 $Y = \{(w_j, t_j)\}$ ，希望计算交集元素的和。
- $H(\cdot)$ ：哈希到群的函数，输出 $g^h \bmod p$ 形式的群元素。
- k_1, k_2 ：双方各自生成的随机私钥指数。
- Enc, Dec ：Paillier 加密与解密。

协议流程 协议可用如下公式与步骤表示：

1. 初始化：

- 双方约定大素数 p 和生成元 g ，构成群 $G = \langle g \rangle$ 。
- P_2 生成 Paillier 公钥 pk 与私钥 sk 。

2. Round 1 ($P_1 \rightarrow P_2$):

$$Z_1 = \{ H(v_i)^{k_1} \bmod p \mid v_i \in X \} \quad (1)$$

P_1 将 Z_1 随机打乱后发送给 P_2 。

3. Round 2 ($P_2 \rightarrow P_1$):

(a) 对每个 $z \in Z_1$:

$$z' = z^{k_2} \bmod p \quad (2)$$

得到集合 $Z_2 = \{z^{k_2}\}$ ，打乱后发给 P_1 。

(b) 对每个 $(w_j, t_j) \in Y$:

$$h'_j = H(w_j)^{k_2} \bmod p \quad (3)$$

$$c_j = \text{Enc}_{\text{pk}}(t_j) \quad (4)$$

得到列表 $\{(h'_j, c_j)\}$ ，打乱后与 Z_2 一并发给 P_1 。

4. Round 3 ($P_1 \rightarrow P_2$):

(a) P_1 对收到的每个 (h'_j, c_j) :

$$h''_j = (h'_j)^{k_1} \bmod p \quad (5)$$

(b) 交集判定:

$$\mathcal{I} = \{j \mid h''_j \in Z_2\} \quad (6)$$

(c) 同态求和:

$$C = \bigoplus_{j \in \mathcal{I}} c_j \quad (7)$$

其中 $\oplus$ 表示 Paillier 同态加法（即密文相乘）。

(d) 重新随机化:

$$C' = \text{Refresh}(C) \quad (8)$$

将 C' 发给 P_2 。

5. 输出:

$$S = \text{Dec}_{\text{sk}}(C') \quad (9)$$

P_2 得到交集对应值的总和 S ，并可统计交集大小 $|\mathcal{I}|$ 。

安全性分析 该协议在随机预言机模型下，基于 DDH 假设和 Paillier 加密的语义安全性，能够保证：

- 非交集元素的信息不泄露。
- 交集求和过程不暴露单个值，仅泄露总和。

6.3 基础模型

可以正确运行：

```
> |
= RESTART: G:\Desktop\2025summer\project6\one.py
intersection size: 2
intersection sum: 21390735411009460934242601621354622365
> |
```

6.4 优化原理

在原始 PoC 基础上，我们针对性能、安全性与可维护性进行了多项优化。以下逐条列出主要优化点，并从**动机**、**实现方式**与**权衡**三个方面进行说明。

1. **安全随机源 secrets 动机**：Python 标准库 `random` 为伪随机，不适合密钥或模随机化 r 的生成。**实现**：所有随机数生成（私钥指数 k_1, k_2 、Paillier 随机因子 r 、打乱顺序）均改用 `secrets.SystemRandom()` 或 `secrets.randbelow()`。**权衡**：运行速度略慢，但随机性安全性显著提高。
2. **素数生成与检测 (Miller-Rabin) 动机**：原代码使用固定或弱检测素数，不安全且不可扩展。**实现**：实现 Miller-Rabin 概率素性测试，结合 `secrets` 随机生成指定位长的素数：

$$\text{is_probable_prime}(n, k) \Rightarrow \Pr[n \text{ 为合数}] \leq 4^{-k}$$

权衡：位长越大生成时间越长，PoC 中可用 128/256 bit 加快测试，生产环境建议 ≥ 2048 bit。

3. **缓存 n^2 提升 Paillier 性能 动机**：Paillier 加密和同态运算频繁使用模 n^2 ，重复计算开销不必要。**实现**：在密钥结构中缓存 n^2 ，后续运算直接引用。**权衡**：节省重复乘法成本，提高可读性。
4. **合并幂运算减少模幂次数 动机**：模幂是协议中最耗时的操作。对于

$$(h^{k_1})^{k_2} \bmod p$$

可改为单次：

$$h^{(k_1 \cdot k_2) \bmod (p-1)} \bmod p$$

利用费马小定理减少一次 `pow` 调用。**实现**：本地模拟时在可得 h 的情况下合并指数，仅做一次模幂。**权衡**：在真实分布式场景中， k_1, k_2 属于不同方私钥，不能直接合并；此优化仅适用于本地仿真或单方持有指数的情况。

5. **集合 (set) 加速成员检测** 动机: 原始交集判定为 $O(nm)$, 数据量大时性能急剧下降。**实现**: 将 Z_2 转为集合 $\text{set}(Z_2)$, 成员检测均摊复杂度降至 $O(1)$ 。**权衡**: 占用更多内存, 但在大规模数据下显著提升速度。
6. **清晰 API 与类型注解** 动机: 增强代码可读性与可维护性, 方便替换底层实现 (如改用标准 ECC 库或成熟 Paillier 库)。**实现**: 将 Paillier 相关函数、群运算、协议逻辑模块化, 并加入 Python 类型注解。**权衡**: 略增代码长度, 但结构更清晰, 测试与维护成本降低。
7. **安全打乱顺序(shuffle)** 动机: 防止交互中元素顺序泄露信息。**实现**: 使用 `secrets.SystemRandom().shuffle()` 替代 `random.shuffle()`。**权衡**: 增加少量延迟, 提升不可预测性。
8. **工程化细节** 动机: 方便测试与参数调整。**实现**: 将群位长、Paillier 位长、哈希种子等作为函数参数; 增加 `__main__` 中的简单测试用例。**权衡**: 无明显性能影响, 提高代码复用性。

性能与安全权衡

- **性能测试**: 可选用较小位长 (如 128/256 bit) 并在本地模拟中合并幂运算以减少模幂次数。
- **安全部署**: 应使用大位长素数 (群 ≥ 2048 bit, Paillier ≥ 2048 bit), 标准群域或椭圆曲线, 以及成熟的 Paillier/ECC 库; 在分布式环境中遵循协议逐步计算, 避免指数合并。

6.5 优化效果

可以看到优化成功:

```
===== RESTART: G:\Desktop\2025summer\project6\test.py =====
=== Summary ===
trials: 4
toy_mean_s: 0.0009906250002131856
opt_mean_s: 0.004260025000121459
match_size_every_run: True
match_sum_every_run: True

=== Per-trial results ===
{'trial': 1, 'toy_time_s': 0.0014048000002730987, 'opt_time_s': 0.0037251000003379886, 'toy_size': 2, 'opt_size': 2, 'toy_sum': 30, 'opt_sum': 30, 'expected_sum': 30, 'toy_paillier_n_bits': 127, 'opt_paillier_n_bits': 255, 'toy_ciphertext_bits': 252, 'opt_ciphertext_bits': 510}
{'trial': 2, 'toy_time_s': 0.0008264999996754341, 'opt_time_s': 0.004716000000371423, 'toy_size': 2, 'opt_size': 2, 'toy_sum': 30, 'opt_sum': 30, 'expected_sum': 30, 'toy_paillier_n_bits': 128, 'opt_paillier_n_bits': 256, 'toy_ciphertext_bits': 255, 'opt_ciphertext_bits': 511}
{'trial': 3, 'toy_time_s': 0.0009810000001380104, 'opt_time_s': 0.004876599999988684, 'toy_size': 2, 'opt_size': 2, 'toy_sum': 30, 'opt_sum': 30, 'expected_sum': 30, 'toy_paillier_n_bits': 128, 'opt_paillier_n_bits': 255, 'toy_ciphertext_bits': 254, 'opt_ciphertext_bits': 508}
{'trial': 4, 'toy_time_s': 0.0007502000007661991, 'opt_time_s': 0.00372239999978774, 'toy_size': 2, 'opt_size': 2, 'toy_sum': 30, 'opt_sum': 30, 'expected_sum': 30, 'toy_paillier_n_bits': 128, 'opt_paillier_n_bits': 255, 'toy_ciphertext_bits': 250, 'opt_ciphertext_bits': 510}
```